

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO CHUYÊN ĐỀ
MÔN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

CHƯƠNG 15: CẤU TRÚC CHUYÊN BIỆT
(SPECIALIZED STRUCTURES)

Nghiên cứu chuyên sâu về Cây tiền tố (Trie) và Cấu trúc tập hợp rời nhau (Union-Find) trong thiết kế hệ thống hiệu năng cao

Giảng viên hướng dẫn:
Thầy Vũ Đình Bảo (T-Bao)

Nhóm sinh viên thực hiện (Lớp DASA230179_06):

1. Lê Thị Tuyết Nhi - MSSV: 25110283
2. Mai Thanh Trà - MSSV: 25110372
3. Huỳnh Thị Thùy Trang - MSSV: 25110371

TP. Hồ Chí Minh, năm 2026

MỤC LỤC

1. PHẦN MỞ ĐẦU VỀ CẤU TRÚC CHUYÊN BIỆT.....	4
1.1. Giới hạn vật lý của các cấu trúc dữ liệu tổng quát	4
1.2. Nguyên lý "Query Pattern First, Structure Second"	4
1.3. Triết lý đánh đổi (Specialization and Trade-off).....	5
1.4. Tác động vật lý của cấu trúc dữ liệu lên phần cứng máy tính.....	6
2. CẤU TRÚC CÂY TIỀN TỔ (TRIE / PREFIX TREE)	6
2.1. Bài toán Autocomplete và Prefix Query	6
2.2. Vì sao Hash Table, Sorted Array và Balanced BST chưa tối ưu	7
2.3. Cấu trúc Trie: Root, Node, Edge và Prefix	7
2.4. Vai trò sinh tử của Cờ kết thúc từ (is_end_of_word).....	8
2.5. Các thuật toán vận hành chính của Trie	8
2.6. Mã giả của các thuật toán Trie	9
2.7. Worked Example: Minh họa từng bước chèn chuỗi	10
2.8. Phân tích độ phức tạp thời gian và không gian	11
2.9. Đánh đổi bộ nhớ và cấu trúc vật chứa nút con (Child Container)	12
2.10. Các lỗi thường gặp và Thực tiễn tốt nhất cho Trie.....	12
2.11. Ứng dụng thực tế của Trie trong công nghiệp.....	13
3. CẤU TRÚC TẬP HỢP KHÔNG GIAO NHAU (UNION-FIND / DSU)	13
3.1. Bài toán kết nối động (Dynamic Connectivity)	13
3.2. Vì sao DFS/BFS hoặc Quick-Find chưa tối ưu.....	14
3.3. Kiểu dữ liệu trừu tượng Disjoint-Set ADT và Triết lý "Cắt bỏ để tăng tốc"	14
3.4. Biểu diễn cấu trúc bằng Mảng cha (Parent Array).....	15
3.5. Trực giác thiết kế "Hội nhóm và Chủ tịch" (Clubs with Presidents)	15
3.6. Phiên bản Naive (Ngây thơ) và sự suy biến cây thành danh sách liên kết ...	16
3.7. Tối ưu hóa 1: Union by Rank (Cơ chế Ngăn ngừa)	16
3.8. Tối ưu hóa 2: Path Compression (Cơ chế Sửa chữa)	17
3.9. Sự kết hợp sinh tử của hai tối ưu hóa và Hàm ngược Ackermann $\alpha(n)$	17

3.10. Mã giả các thuật toán Union-Find.....	18
3.11. Worked Example: Minh họa từng bước và Bảng mô phỏng vết (Trace Table)	20
3.12. Phân tích mở rộng: Tối ưu hóa Union by Size (Gộp theo kích thước)	21
3.13. Các lỗi thường gặp khi triển khai Union-Find	22
3.14. Ứng dụng thực tế của Union-Find.....	23
4. PHÂN TÍCH SO SÁNH VÀ ĐÁNH GIÁ ĐỐI SÁNH.....	23
4.1. Bài toán xử lý chuỗi và từ điển: Trie vs Hash Table vs Balanced BST.....	23
4.2. Bài toán liên thông đồ thị: Union-Find vs BFS/DFS	24
4.3. So sánh triết lý thiết kế tổng thể giữa Trie và Union-Find.....	25
4.4. Khung ra quyết định kỹ thuật toàn diện (Technical Decision Framework)..	26
5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN MỞ RỘNG	27
5.1. Tóm tắt kết quả nghiên cứu	27
5.2. Hướng phát triển mở rộng cho Trie.....	27
5.3. Hướng phát triển mở rộng cho Union-Find.....	27
6. TÀI LIỆU THAM KHẢO (REFERENCES).....	28

1. PHẦN MỞ ĐẦU VỀ CẤU TRÚC CHUYÊN BIỆT

Trong suốt quá trình đào tạo khoa học máy tính và thực hành phần mềm cơ bản, lập trình viên thường được trang bị một "hộp công cụ" tiêu chuẩn bao gồm các cấu trúc dữ liệu kinh điển: Mảng (Arrays), Danh sách liên kết (Linked Lists), Ngăn xếp (Stacks), Hàng đợi (Queues), Bảng băm (Hash Tables), Cây tìm kiếm nhị phân (Binary Search Trees), và Heap. Đây là các **cấu trúc dữ liệu tổng quát (general-purpose structures)**, được thiết kế để giải quyết hai câu hỏi bao quát nhất của việc lưu trữ thông tin: "Lưu trữ cái gì?" và "Tìm kiếm lại nó như thế nào?". Ưu điểm lớn nhất của chúng là tính linh hoạt, đa dụng và khả năng tái sử dụng cao trong nhiều ngữ cảnh lập trình thông thường.

1.1. Giới hạn vật lý của các cấu trúc dữ liệu tổng quát

Tuy nhiên, khi đối mặt với các bài toán có quy mô dữ liệu cực lớn trong môi trường công nghiệp hiện đại, các công cụ tổng quát này bắt đầu bộc lộ các giới hạn vật lý nghiêm trọng. Điểm nghẽn không nằm ở chỗ chương trình bị sụp đổ hoàn toàn (crash), mà nằm ở sự suy giảm hiệu năng nghiêm trọng khi dữ liệu tăng trưởng vượt qua ngưỡng tối ưu. Các cấu trúc tổng quát này cố gắng làm tốt mọi thứ, dẫn đến việc chúng không thể đạt được tốc độ tối đa cho bất kỳ một thao tác chuyên biệt nào.

Ví dụ, một bảng băm có thể tìm kiếm phần tử chính xác cực kỳ xuất sắc với độ phức tạp thời gian trung bình là $O(1)$, nhưng nó lại hoàn toàn bất lực và phải quét toàn bộ dữ liệu $O(n)$ khi được hỏi về các quan hệ tiền tố hoặc các tập hợp con liên kết. Tương tự, các cây tìm kiếm cân bằng duy trì cấu trúc động rất tốt nhưng lại gánh chịu chi phí ẩn vô cùng lớn về con trỏ và các phép so sánh chuỗi phức tạp tại mỗi tầng của cây. Sự suy giảm hiệu năng này đòi hỏi chúng ta phải thay đổi tư duy thiết kế từ gốc rễ.

1.2. Nguyên lý "Query Pattern First, Structure Second"

Triết lý thiết kế quan trọng nhất làm nền tảng cho toàn bộ chuyên đề nghiên cứu cấu trúc dữ liệu chuyên biệt được đúc kết ngắn gọn qua nguyên lý: **"Look at the query pattern first, then choose the structure"** (Xem xét mẫu truy vấn trước, chọn cấu

trúc dữ liệu sau). Các lập trình viên thông thường có xu hướng nhồi nhét dữ liệu vào một cấu trúc quen thuộc hiện có (như mảng hoặc bảng băm) rồi sau đó cố gắng viết các giải thuật tìm kiếm phức tạp để trích xuất thông tin. Tư duy này đi ngược lại với kiến trúc phần mềm tối ưu. Một kiến trúc sư hệ thống thực thụ sẽ phân tích xem mã nguồn thực sự hỏi câu hỏi gì nhiều nhất và lập đi lập lại với tần suất cao nhất.

Nếu câu hỏi mang tính chất rộng và tổng quát ("hãy lưu trữ đối tượng này và sau đó tìm lại nó một cách chính xác"), một cấu trúc tổng quát là lựa chọn tối ưu nhất. Nhưng nếu câu hỏi là một dạng truy vấn cực kỳ hẹp và có cấu trúc đặc thù ("những từ khóa nào bắt đầu bằng bốn chữ cái này?" hoặc "hai tài khoản này có thuộc chung một chủ thể liên kết hay không?"), một cấu trúc dữ liệu chuyên biệt (purpose-built/specialized structure) được thiết kế đo ni đóng giày cho câu hỏi hẹp đó sẽ mang lại hiệu năng vượt trội, bỏ xa các công cụ truyền thống.

1.3. Triết lý đánh đổi (Specialization and Trade-off)

Một cấu trúc dữ liệu chuyên biệt không cố gắng trở thành một giải pháp đa năng "vạn năng". Sức mạnh và tốc độ vượt trội của chúng có được là nhờ vào việc tuân thủ một triết lý đánh đổi (trade-off) cực kỳ chủ động và táo bạo:

TỐC ĐỘ CỰC HẠN CHO TRUY VẤN HẸP ↔ SỰ TỪ BỎ CHỦ ĐỘNG CÁC TÍNH NĂNG TỔNG QUÁT KHÁC

Nhờ vào việc xác định rõ ràng câu hỏi hẹp cần giải quyết, cấu trúc chuyên biệt sẵn sàng loại bỏ những tính năng dư thừa để dồn toàn bộ tài nguyên tối ưu hóa cho mục tiêu duy nhất đó:

- **Cây tiền tố (Trie):** Sẵn sàng đánh đổi một lượng lớn tài nguyên bộ nhớ (sử dụng nhiều nút và một ma trận con trỏ liên kết con dày đặc) để đổi lấy khả năng tìm kiếm tiền tố siêu tốc có độ phức tạp thời gian $O(L)$ chỉ phụ thuộc duy nhất vào chiều dài của từ khóa L , hoàn toàn độc lập với tổng quy mô từ điển n đang lưu trữ.
- **Tập hợp không giao nhau (Union-Find):** Sẵn sàng loại bỏ hoàn toàn khả năng liệt kê các thành viên trong một nhóm hoặc khả năng hoàn tác (undo/disconnect) thao tác gộp nhóm. Bằng việc từ bỏ các tính năng không cần thiết này, Union-Find không

cần duy trì danh sách liên kết ngược phức tạp, từ đó đạt được tốc độ gộp nhóm và truy vấn kết nối siêu hằng số $O(\alpha(n))$ hầu như tuyệt đối bằng cách sử dụng duy nhất một mảng số nguyên đơn giản.

1.4. Tác động vật lý của cấu trúc dữ liệu lên phần cứng máy tính

Dưới góc nhìn của kỹ nghệ phần cứng và kiến trúc máy tính hiện đại (Hardware-aware Software Engineering), sự bế tắc của các cấu trúc dữ liệu tổng quát trên quy mô dữ liệu cực lớn còn xuất phát từ một khái niệm vật lý cốt lõi: **Tính cục bộ của bộ nhớ (Memory Locality)** và **hiện tượng hụt bộ đệm ẩn (Cache Misses)**.

Khi một bảng băm thực hiện băm một khóa, nó sẽ ánh xạ khóa đó tới một vị trí ngẫu nhiên trên thanh RAM. Đối với các tập dữ liệu nhỏ nằm trọn trong bộ bộ nhớ đệm L1/L2 của CPU, điều này không gây ra vấn đề gì. Nhưng khi quy mô đạt hàng triệu phần tử, mỗi phép truy cập bảng băm sẽ ép CPU phải nhảy tới các địa chỉ nhớ phân tán ngẫu nhiên, gây ra hiện tượng Cache Miss liên tục. Chi phí thực tế của việc truy xuất RAM vật lý (RAM latency) lớn hơn hàng trăm lần so với tốc độ tính toán của bộ vi xử lý.

Ngược lại, Union-Find loại bỏ hoàn toàn việc cấp phát bộ nhớ động thông qua việc biểu diễn rừng cây trực tiếp trên một mảng một chiều liên tục duy nhất (parent). Việc truy xuất mảng tuần tự giúp CPU tận dụng tối đa cơ chế nạp trước bộ đệm (spatial locality), giảm thiểu Cache Miss xuống mức tối đa. Đây chính là lý do vì sao Union-Find có tốc độ thực thi trong thực tế cực kỳ đáng kinh ngạc, vượt xa các giải thuật đồ thị dựa trên con trỏ và danh sách kề truyền thống.

2. CẤU TRÚC CÂY TIỀN TỔ (TRIE / PREFIX TREE)

2.1. Bài toán Autocomplete và Prefix Query

Hệ thống tìm kiếm hiện đại đòi hỏi tốc độ phản hồi tức thời. Khi người dùng nhập một chuỗi ký tự ngắn vào ô tìm kiếm, hệ thống phải thực hiện **Prefix Query (truy vấn theo tiền tố)** để đưa ra danh sách các từ gợi ý hoàn chỉnh (Autocomplete). Ví dụ, nếu người dùng nhập "lap", hệ thống phải nhanh chóng trả về "laptop", "lapel",

"laptop sleeve" từ một kho cơ sở dữ liệu hàng triệu từ khóa trước khi phím tiếp theo được nhấn.

2.2. Vì sao Hash Table, Sorted Array và Balanced BST chưa tối ưu

Khi đối phó với bài toán Autocomplete, ba cấu trúc dữ liệu tổng quát thông dụng nhất đều bộc lộ những khuyết điểm nghiêm trọng về thuật toán hoặc cấu trúc vật lý:

- Bảng băm (Hash Table) - Sự lệch pha hoàn toàn về mặt cấu trúc hình học: Hàm băm băm chuỗi thành chỉ số ngẫu nhiên để phân bố đều dữ liệu, phá vỡ mọi mối quan hệ tuần tự giữa các chuỗi ký tự. Hai chuỗi có chung tiền tố rất dài như "lap" và "laptop" bị băm thành hai giá trị hoàn toàn độc lập, nằm ở các bucket xa lạ trong RAM. Do đó, muốn tìm các từ có tiền tố "lap", bảng băm bắt buộc phải thực hiện quét toàn bộ (full scan) toàn bộ n từ khóa với độ phức tạp cực thảm họa $O(n * L)$.

- Mảng đã sắp xếp (Sorted Array) - Rào cản ghi dữ liệu động: Sắp xếp mảng giúp gom các từ có chung tiền tố vào một khối liên tục, cho phép sử dụng tìm kiếm nhị phân để định vị trong thời gian $O(L \log n)$. Tuy nhiên, nếu từ điển cập nhật động liên tục, việc chèn một từ mới đòi hỏi phải dịch chuyển các phần tử đứng sau, tốn chi phí $O(n)$ cho mỗi lần chèn.

- Cây tìm kiếm nhị phân cân bằng (Balanced BST) - Chi phí so sánh chuỗi ẩn: AVL hay Đỏ-Đen giải quyết được thao tác ghi động nhưng phép so sánh khóa tại mỗi nút tốn $O(L)$. Khi đi xuống cây qua chiều cao $h \approx \log n$, tổng chi phí tìm kiếm đạt $O(L \log n)$. Nhân tử L trực tiếp nhân vào $\log n$ gây sụt giảm hiệu năng rất lớn đối với chuỗi dài.

2.3. Cấu trúc Trie: Root, Node, Edge và Prefix

Trie (viết tắt của Re-trie-val, phát âm là "try") phá vỡ thói quen coi chuỗi là khối dữ liệu nguyên khối. Trie là một cây k-ngôi có gốc, với các đặc điểm cấu trúc độc đáo:

1. Nút của Trie không lưu trữ ký tự hay khóa dữ liệu. Nút chỉ đóng vai trò điểm giao lộ kết nối các hướng đi tiếp theo.

2. Ký tự nằm trên cạnh nối: Ký tự chính là nhãn dán trên các cạnh liên kết từ nút cha xuống nút con.

3. Khóa nằm trên đường đi: Chuỗi khóa được hình thành gián tiếp bằng cách tích lũy các ký tự trên các cạnh dọc theo đường đi từ gốc đến nút đó.

2.4. Vai trò sinh tử của Cờ kết thúc từ (**is_end_of_word**)

Do các từ có thể chứa nhau làm tiền tố (ví dụ "car" và "cart"), Trie bắt buộc phải duy trì một cờ boolean mang tên **is_end_of_word** (hoặc **is_end**) tại mỗi nút. Cờ này ghi nhận đường đi kết thúc tại nút hiện tại đại diện cho một khóa hoàn chỉnh thực sự được lưu trữ, chứ không chỉ là tiền tố của một từ khác dài hơn. Nếu không có cờ này, từ "car" sẽ bị nuốt chửng hoàn toàn trong từ "cart" và biến mất khỏi hệ thống. Nó hoạt động giống như một tấm thảm chùi chân (doormat) đặt trước cửa phòng: đạt tới nút nghĩa là đường đi tồn tại, nhìn thấy tấm thảm mới chứng minh đây là một đích đến hợp lệ.

2.5. Các thuật toán vận hành chính của Trie

- Thao tác Chèn từ (Insert): Bắt đầu từ gốc, duyệt qua từng ký tự của từ. Nếu cạnh nối mang nhãn ký tự chưa tồn tại, khởi tạo nút mới và dựng cạnh nối. Cuối cùng, bật cờ **is_end_of_word** = true tại nút đích. Độ phức tạp: $O(L)$.
- Thao tác Tìm kiếm chính xác (Search): Đi dọc xuống từ gốc theo các ký tự của từ. Nếu bất kỳ cạnh nào bị thiếu, trả về false. Khi đi hết từ, kiểm tra cờ **is_end_of_word**; nếu cờ bật thì từ tồn tại, ngược lại trả về false. Độ phức tạp: $O(L)$.
- Thao tác Kiểm tra tiền tố (startsWith): Hoàn toàn tương tự Search nhưng không cần kiểm tra cờ **is_end_of_word**. Đặt chân được tới nút cuối cùng là đủ để kết luận tiền tố tồn tại. Độ phức tạp: $O(L)$.
- Thao tác Liệt kê từ theo tiền tố (wordsWithPrefix): Trái tim của tính năng Autocomplete. Giai đoạn 1: đi bộ từ gốc để định vị nút tiền tố trong $O(L)$. Giai đoạn 2: thực hiện duyệt sâu (DFS) trên cây con bên dưới nút tiền tố đó để thu thập và tái cấu trúc lại tất cả các từ hoàn chỉnh.

2.6. Mã giả của các thuật toán Trie

```
class TrieNode:
    constructor():
        children = Dictionary() # Ánh xạ character -> TrieNode
        is_end_of_word = false

class Trie:
    constructor():
        root = new TrieNode()

    function insert(word):
        node = root
        for each char in word:
            if char not in node.children:
                node.children[char] = new TrieNode()
            node = node.children[char]
        node.is_end_of_word = true

    function walk(prefix):
        node = root
        for each char in prefix:
            if char not in node.children:
                return null
            node = node.children[char]
        return node

    function search(word):
```

```

node = walk(word)
return (node != null) and node.is_end_of_word

function starts_with(prefix):
    return walk(prefix) != null

function words_with_prefix(prefix):
    results = empty list
    start_node = walk(prefix)
    if start_node != null:
        collect(start_node, prefix, results)
    return results

function collect(node, current_string, results):
    if node.is_end_of_word:
        results.append(current_string)
    for each (char, child_node) in node.children:
        collect(child_node, current_string + char, results)

```

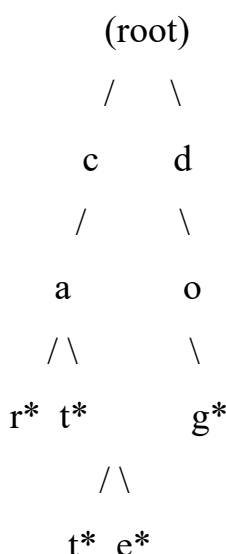
2.7. Worked Example: Minh họa từng bước chèn chuỗi

Chúng ta sẽ thực hiện chèn lần lượt 5 khóa sau vào một Trie rỗng: **car**, **cart**, **care**, **cat**, **dog**.

- Bước 1 (car): Bắt đầu từ gốc rỗng. Tạo chuỗi nút liên tiếp: c -> a -> r. Tại nút r, bật cờ kết thúc (car*).
- Bước 2 (cart): Đi từ gốc theo đường c -> a -> r có sẵn, tại nút r tạo thêm nút t và bật cờ kết thúc (cart*).

- Bước 3 (care): Tái sử dụng c -> a -> r có sẵn, tại nút r rẽ nhánh mới tạo nút e và bật cờ kết thúc (care*).
- Bước 4 (cat): Tái sử dụng c -> a, tại nút a rẽ nhánh mới tạo nút t và bật cờ kết thúc (cat*).
- Bước 5 (dog): Hoàn toàn không chia sẻ tiền tố với các từ trước, khởi tạo nhánh độc lập từ gốc: d -> o -> g và bật cờ kết thúc (dog*).

Sơ đồ cấu trúc cây Trie sau khi chèn hoàn tất:



Ký hiệu: * đại diện cho cờ is_end_of_word = true

2.8. Phân tích độ phức tạp thời gian và không gian

- Độ phức tạp thời gian: Thao tác chèn, tìm kiếm chính xác và kiểm tra tiền tố có độ phức tạp thời gian cực hạn là **O(L)** trong trường hợp xấu nhất, với L là chiều dài của từ khóa đang xử lý. Chi phí này hoàn toàn độc lập với tổng số lượng từ khóa n đang lưu trên cây. Một cây chứa 5 từ hay chứa 5 triệu từ đều tốn số bước chân như nhau để tìm kiếm một từ có độ dài 5 ký tự. Đây là ưu thế tuyệt đối giúp Trie vượt trội hơn hẳn mọi cấu trúc tổng quát khi quy mô dữ liệu n phình to.

- Độ phức tạp không gian: Trong trường hợp xấu nhất, độ phức tạp không gian đạt mức $O(n * L * |\Sigma|)$ khi tất cả các từ khóa hoàn toàn không chia sẻ tiền tố, buộc hệ thống phải khởi tạo nút mới cho từng ký tự một và duy trì vật chứa các con trỏ con kích thước $|\Sigma|$. Tuy nhiên, trong ngôn ngữ tự nhiên thực tế, mật độ chia sẻ tiền tố của từ vựng là vô cùng cao, giúp giảm thiểu đáng kể số lượng nút thực tế cần cấp phát.

2.9. Đánh đổi bộ nhớ và cấu trúc vật chứa nút con (Child Container)

Mỗi nút trong Trie đòi hỏi phải duy trì một cấu trúc dữ liệu quản lý các con trỏ con (child container). Có hai giải pháp thiết kế chính với sự đánh đổi sâu sắc:

- Mảng cố định kích thước A (Fixed Array of size A): Mỗi nút giữ một mảng tĩnh kích thước bằng kích thước bảng chữ cái (ví dụ $A=26$ đối với ký tự tiếng Anh thường). Ưu điểm: Tốc độ tối đa $O(1)$ cho mỗi bước nhảy. Nhược điểm: Lãng phí bộ nhớ vô cùng khủng khiếp. Nếu nút chỉ có một cạnh con thực tế, nó vẫn phải duy trì $(A - 1)$ ô con trỏ rỗng (null). Trong bảng mã Unicode (A lên tới hàng chục nghìn), giải pháp này gây tràn bộ nhớ ngay lập tức.
- Bản đồ băm động (Hash Map / Dynamic Map): Mỗi nút duy trì một bảng băm, chỉ cấp phát bộ nhớ cho các cạnh con thực tế tồn tại. Ưu điểm: Tiết kiệm tối đa tài nguyên bộ nhớ, linh hoạt hỗ trợ Unicode. Nhược điểm: Tốn thêm một chút chi phí thời gian để thực hiện phép tính toán hàm băm cho ký tự tại mỗi bước nhảy.

2.10. Các lỗi thường gặp và Thực tiễn tốt nhất cho Trie

- Ba sai lầm kinh điển của lập trình viên:

1. Lầm tưởng nút lưu trữ ký tự hoặc từ: Nút thực tế hoàn toàn trống rỗng, ký tự nằm ở cạnh và khóa nằm ở đường đi. Việc gán trường ký tự vào trong nút là dư thừa.
2. Lầm tưởng đường đi tồn tại nghĩa là từ tồn tại: Việc đi bộ thành công chỉ chứng minh chuỗi đó là tiền tố. Bắt buộc phải kiểm tra cờ kết thúc từ để xác định thành viên.
3. Lạm dụng Trie cho bài toán tìm kiếm chính xác (exact-match): Nếu không có yêu cầu truy vấn tiền tố, bảng băm sẽ luôn nhanh hơn và tiết kiệm bộ nhớ gấp hàng chục

lần vì nó tránh được việc nhảy con trỏ liên tục trên bộ nhớ phân tán.

- Ba nguyên tắc thiết kế vàng:

1. Chuẩn hóa khóa (Normalization) trước khi thao tác: Luôn đưa toàn bộ chuỗi về dạng chữ thường, cắt khoảng trắng và loại bỏ dấu để tránh việc tạo ra các nhánh thừa không cần thiết do Trie phân biệt ký tự tuyệt đối về mặt vật lý.

2. Đặt giới hạn kết quả trả về (Capping Results): Một truy vấn tiền tố siêu ngắn như chữ 'a' có thể quét sạch và trả về 80% từ điển lớn, gây ngẽn RAM. Luôn đặt một ngưỡng giới hạn (ví dụ: dừng ngay khi tìm đủ 5 hoặc 10 gợi ý).

3. Đo đạc kỹ lưỡng trước khi triển khai: Nếu số lượng truy vấn tiền tố chiếm dưới 5% tổng số truy vấn, việc duy trì một cấu trúc Trie phức tạp là không thực sự mang lại lợi ích kinh tế cho hệ thống.

2.11. Ứng dụng thực tế của Trie trong công nghiệp

- Hệ thống bàn phím dự đoán từ thông minh trên thiết bị di động: Gboard, Apple Keyboard tổ chức từ điển dưới dạng Trie để tìm kiếm tức thì các từ gợi ý phù hợp nhất trong thời gian thực, đảm bảo hiệu năng phản hồi tức thời, loại bỏ hoàn toàn độ trễ giao diện.

- Bảng định tuyến địa chỉ IP trong thiết bị mạng cốt lõi (Routers): Thao tác tìm kiếm công ra định tuyến theo luật Longest Prefix Match (So khớp tiền tố dài nhất) được giải quyết tối ưu bằng cách biểu diễn dải mạng dưới dạng một Trie nhị phân ($A=2$ đại diện cho 0 và 1), giúp router kiểm tra so khớp tiền tố chỉ bằng cách duyệt dọc xuống cây theo các bit của địa chỉ IP đích.

3. CẤU TRÚC TẬP HỢP KHÔNG GIAO NHAU (UNION-FIND / DSU)

3.1. Bài toán kết nối động (Dynamic Connectivity)

Trong phát triển phần mềm và lý thuyết đồ thị, bài toán kết nối động (Dynamic Connectivity) yêu cầu quản lý một hệ thống mạng lưới liên tục biến đổi (add-only) theo thời gian. Tại mọi thời điểm, hệ thống đối mặt với hai loại yêu cầu xuất hiện đan xen liên tục với tần suất cực cao:

1. Yêu cầu Liên kết (Link/Union): Thiết lập kết nối vật lý hoặc logic giữa hai thực thể độc lập (ví dụ: gộp hai tài khoản trùng lặp, hoặc kết nối hai điểm nút trong mạng lưới truyền thông).

2. Yêu cầu Truy vấn (Connected): Kiểm tra xem hai thực thể bất kỳ có liên thông trực tiếp hoặc gián tiếp với nhau qua chuỗi liên kết đã được thiết lập từ trước đến nay hay không.

3.2. Vì sao DFS/BFS hoặc Quick-Find chưa tối ưu

- Duyệt đồ thị trực tiếp khi được hỏi (DFS/BFS trên Danh sách kề): Thiết lập liên kết diễn ra rất nhanh $O(1)$ nhưng mỗi truy vấn connected phải chạy DFS/BFS để dò đường đi từ nút nguồn đến nút đích, tốn chi phí $O(V+E)$ (hoặc $O(n)$). Một chuỗi m truy vấn sẽ đẩy tổng chi phí lên $O(m * n)$, hoàn toàn bất khả thi cho các hệ thống thời gian thực.

- Gán nhãn nhóm trực tiếp (Quick-Find): Lưu giữ nhãn nhóm của từng phần tử. Truy vấn connected cực nhanh $O(1)$ nhờ so sánh trực tiếp hai nhãn nhóm. Tuy nhiên, khi gộp hai nhóm (union), hệ thống bắt buộc phải duyệt qua toàn bộ mảng dữ liệu để cập nhật nhãn mới cho tất cả các thành viên của nhóm bị gộp, tốn chi phí $O(n)$. Thực hiện một chuỗi n thao tác gộp liên tiếp sẽ kéo tổng chi phí lên mức thảm họa lũy thừa $O(n^2)$.

3.3. Kiểu dữ liệu trừu tượng Disjoint-Set ADT và Triết lý "Cắt bỏ để tăng tốc"

Để giải quyết triệt để sự bế tắc của các giải pháp truyền thống, Kiểu dữ liệu trừu tượng Tập hợp không giao nhau (Disjoint-Set ADT), hay cấu trúc **Union-Find (DSU)**, được định nghĩa để quản lý một tập hợp các phần tử được phân chia thành các nhóm (tập con) không giao nhau. Union-Find cam kết hỗ trợ ba thao tác nền tảng:

- `make_set(x)`: Khởi tạo một tập hợp mới chứa duy nhất phần tử x .
- `find(x)`: Truy vết và trả về định danh duy nhất (phần tử đại diện tối cao) của nhóm chứa phần tử x .
- `union(a, b)`: Hợp nhất toàn bộ nhóm chứa phần tử a và nhóm chứa phần tử b . Thao tác tiện ích `connected(a, b)` được xây dựng trực tiếp: `find(a) == find(b)`.

Triết lý cốt lõi của Union-Find chính là chủ động từ bỏ các tính năng không cần thiết: không có thao tác liệt kê tất cả các thành viên trong một nhóm, và không có thao tác xóa kết nối hoặc tách nhóm (undo/disconnect). Nhờ việc giải phóng bản thân khỏi gánh nặng quản lý các danh sách con trở thành viên phức tạp này, Union-Find đạt được tốc độ xử lý siêu hằng số trong thực tế.

3.4. Biểu diễn cấu trúc bằng Mảng cha (Parent Array)

Toàn bộ cấu trúc rừng cây được biểu diễn tinh gọn bằng duy nhất một mảng số nguyên parent kích thước n . Vị trí chỉ mục $\text{parent}[x]$ lưu giữ chỉ số của phần tử cấp trên trực tiếp mà x báo cáo lên. Phần tử đại diện tối cao (gốc) của nhóm chính là một nút tự trở vào chính nó trong mảng: $\text{parent}[x] == x$. Thao tác $\text{find}(x)$ là hành động đi dọc theo các mối liên kết cha $\text{parent}[x]$ ngược lên trên cho đến khi gặp một nút tự trở.

★ CẢNH BÁO QUAN TRỌNG: Cây trong rừng Union-Find hoàn toàn KHÔNG phải là cây tìm kiếm nhị phân (BST). Không có quy luật thứ tự lớn hơn hay nhỏ hơn giữa cha và con; không có duyệt cây theo thứ tự; và mũi tên liên kết trở ngược từ dưới lên (từ con lên cha). Do hình dạng cây không chứa thông tin đặc trưng nào của phần tử nên chúng ta hoàn toàn tự do thay đổi, kéo dẹt hoặc làm phẳng cấu trúc cây bất cứ lúc nào để tối ưu hóa hiệu năng.

3.5. Trực giác thiết kế "Hội nhóm và Chủ tịch" (Clubs with Presidents)

Hãy tưởng tượng một căn phòng lớn gồm nhiều câu lạc bộ độc lập. Mỗi câu lạc bộ tự bầu ra một người đại diện tối cao gọi là "Chủ tịch" (nút gốc của cây). Mỗi thành viên chỉ cần ghi nhớ duy nhất một thông tin: ai là cấp trên trực tiếp của mình. Vị Chủ tịch tối cao của nhóm không báo cáo cho ai khác ngoài chính mình.

- Kiểm tra cùng nhóm: Alice và Dana lần lượt đi dọc theo chuỗi báo cáo lên cấp trên cho đến khi gặp vị Chủ tịch tối cao của họ. Nếu gặp chung một vị Chủ tịch, họ thuộc cùng một câu lạc bộ, ngược lại họ thuộc hai nhóm độc lập.
- Gộp nhóm (Union): Khi gộp hai câu lạc bộ, hai vị Chủ tịch gặp nhau và thực hiện một cái bắt tay thỏa thuận: Chủ tịch của nhóm thứ nhất đồng ý báo cáo dưới quyền

Chủ tịch của nhóm thứ hai. Chỉ có duy nhất một mối liên kết của vị Chủ tịch nhóm một bị thay đổi trong mảng parent. Tuy nhiên, từ khoảnh khắc đó, tất cả thành viên của cả hai nhóm khi đi ngược lên đều tự động gặp chung một vị Chủ tịch tối cao mới. Một người thay đổi, hàng triệu thành viên được cập nhật một cách gián tiếp và tức thời.

3.6. Phiên bản Naive (Ngây thơ) và sự suy biến cây thành danh sách liên kết

Trong phiên bản thô sơ nhất (Naive version), thao tác union(a, b) được thực hiện một cách ngây thơ bằng quy tắc tùy ý: gán trực tiếp gốc của cây thứ nhất trở vào gốc của cây thứ hai. Sự ngây thơ này lập tức dẫn đến một thảm họa về mặt hình học. Nếu ta thực hiện một chuỗi gộp liên kết tăng dần như gộp(0,1), gộp(1,2), gộp(2,3)...., hệ thống sẽ tạo ra một cây bị suy biến hoàn toàn thành danh sách liên kết đơn (Cây đội lốt danh sách liên kết) với chiều cao đạt mức tối đa là $n - 1$. Khi đó, mỗi khi gọi find(0) hay connected(0, n-1), thuật toán buộc phải duyệt qua toàn bộ chuỗi tuyến tính này, tốn chi phí $O(n)$. Bài toán hiệu năng lại rơi vào ngõ cụt ban đầu.

3.7. Tối ưu hóa 1: Union by Rank (Cơ chế Ngăn ngừa)

Để giải quyết triệt để lỗi suy biến, chúng ta áp dụng tối ưu hóa thứ nhất mang tên Gộp theo cấp hạng (Union by Rank). Đây là một cơ chế phòng ngừa chủ động (prevention) nhằm kiểm soát nghiêm ngặt chiều cao của cây ngay trong quá trình ghi dữ liệu (union).

Ý tưởng cốt lõi là duy trì thêm một mảng số nguyên rank[x] cho mỗi nút gốc, đại diện cho giới hạn trên về mặt chiều cao của cây có gốc tại x. Khi hợp nhất hai nhóm, ta luôn treo gốc của cây có rank thấp hơn (cây lùn hơn) dưới gốc của cây có rank cao hơn (cây cao hơn). Nhờ quy tắc này, chiều cao của cây lớn hơn sẽ hoàn toàn không thay đổi sau khi gộp. Chỉ trong trường hợp duy nhất khi hai cây có rank hoàn toàn bằng nhau gộp lại, chúng ta mới chọn một bên làm con bên kia, và tiến hành tăng rank của gốc mới lên đúng 1 đơn vị. Cơ chế này đảm bảo chiều cao của mọi cây trong hệ thống Union by Rank luôn được khống chế không bao giờ vượt quá mức

$O(\log n)$. Thao tác find lúc này được không chế nghiêm ngặt trong giới hạn logarithmic ở trường hợp xấu nhất.

3.8. Tối ưu hóa 2: Path Compression (Cơ chế Sửa chữa)

Nếu như Union by Rank là cơ chế phòng ngự từ xa, thì tối ưu hóa thứ hai mang tên Nén đường đi (Path Compression) đóng vai trò là một cơ chế sửa chữa tại chỗ (repair) vô cùng mạnh mẽ trong quá trình đọc dữ liệu (find). Mỗi khi hàm find(x) hoạt động, nó walks từ x lên nút gốc đại diện. Trên con đường đi lên đó, nó gián tiếp biết được gốc tối cao của tất cả các nút trung gian mà nó đi qua. Thay vì vứt bỏ thông tin quý giá này, thuật toán thực hiện dọn dẹp cấu trúc cây ngay lập tức.

Về mặt cơ chế vận hành, hàm find thực hiện qua hai lượt quét (two-pass) liên tiếp:

- Lượt 1 (Climb up): Đi từ nút xuất phát x ngược lên trên theo các liên kết parent để xác định được nút gốc tối cao đại diện cho nhóm.
- Lượt 2 (Flatten path): Quét ngược lại con đường vừa đi qua một lần nữa, ngắt đứt liên kết cũ của tất cả các nút trung gian dọc đường và trở trực tiếp liên kết parent của chúng về nút gốc tối cao vừa tìm thấy ở Lượt 1.

Kết quả là toàn bộ chuỗi đường đi dài ngoằn ngoèo lập tức bị làm phẳng thành một cấu trúc hình sao dẹt có chiều cao bằng 1. Tất cả các nút này giờ đây đều kết nối trực tiếp với nút gốc chỉ trong một bước nhảy duy nhất. Điều này đem lại hiệu quả vô cùng to lớn: các thao tác find tiếp theo trên bất kỳ nút nào trong số này ở tương lai sẽ lập tức đạt được chi phí thời gian hằng số tuyệt đối $O(1)$.

3.9. Sự kết hợp sinh tử của hai tối ưu hóa và Hàm ngược Ackermann $\alpha(n)$

Sức mạnh thực sự của Union-Find chỉ được giải phóng tối đa khi chúng ta kết hợp đồng thời cả hai tối ưu hóa Union by Rank và Path Compression. Chúng không phải là hai giải pháp thay thế hay cạnh tranh nhau, mà là hai đối tác bổ trợ hoàn hảo: Union by Rank ngăn chặn không cho các cây có chiều cao lớn hình thành ngay từ đầu trong thao tác ghi (union); Path Compression làm phẳng ngay lập tức các nhánh cây vô tình bị kéo dài trong quá trình duyệt đọc dữ liệu (find).

Khi kết hợp đồng thời cả hai, chi phí thời gian trung bình cho mỗi thao tác (bao gồm khởi tạo, gộp nhóm và kiểm tra kết nối) giảm xuống mức cực hạn: **$O(\alpha(n))$ khấu hao (amortized complexity)**. Hàm ngược Ackermann $\alpha(n)$ là một hàm toán học có tốc độ tăng trưởng chậm một cách kinh ngạc. Đối với bất kỳ giá trị n nào tồn tại trong thế giới vật lý thực tế — kể cả tổng số lượng hạt nguyên tử cấu thành nên toàn bộ vũ trụ khả kiến — giá trị của hàm $\alpha(n)$ cũng luôn luôn nhỏ hơn hoặc bằng 5. Do đó, trong thực tế kỹ nghệ phần mềm, chi phí $O(\alpha(n))$ được coi là hằng số thực tế (effectively constant).

3.10. Mã giả các thuật toán Union-Find

```
class UnionFind:
    parent = array of size n
    rank = array of size n

    function make_sets(n):
        for i from 0 to n - 1:
            parent[i] = i # Tự trở vào chính nó nghĩa là "Tôi là nút gốc"
            rank[i] = 0

    function find(x):
        # Lượt 1: Leo lên tìm gốc tối cao
        root = x
        while parent[root] != root:
            root = parent[root]

        # Lượt 2: Nén đường đi (Path Compression)
        while parent[x] != root:
            next_node = parent[x]
```

```

    parent[x] = root
    x = next_node

return root

function union(a, b):
    root_a = find(a)
    root_b = find(b)
    if root_a == root_b:
        return false # Đã thuộc cùng tập hợp, không có thao tác gộp thực tế

    # Union by Rank: Treo cây thấp dưới gốc cây cao hơn
    if rank[root_a] < rank[root_b]:
        parent[root_a] = root_b
    else if rank[root_a] > rank[root_b]:
        parent[root_b] = root_a
    else:
        parent[root_b] = root_a
        rank[root_a] = rank[root_a] + 1

return true

function connected(a, b):
    return find(a) == find(b)

```

3.11. Worked Example: Minh họa từng bước và Bảng mô phỏng vết (Trace Table)

Chúng ta sẽ theo dõi chi tiết kịch bản trace trạng thái biến đổi vật lý của mảng parent và mảng cấp hạng rank gồm 7 phần tử từ 0 đến 6 khi có cả hai tối ưu hóa hoạt động đồng thời. Ban đầu, các rank đều bằng 0.

Bước	Thao tác	Mảng parent	Mảng rank	Cơ chế diễn ra bên dưới
0	Khởi tạo ban đầu	[0, 1, 2, 3, 4, 5, 6]	[0, 0, 0, 0, 0, 0, 0]	Mọi phần tử tự trở vào chính mình, rank toàn bộ bằng 0.
1	gộp(0, 1)	[0, 0, 2, 3, 4, 5, 6]	[1, 0, 0, 0, 0, 0, 0]	Rank bằng nhau (0). Treo 1 dưới 0. Rank gốc 0 tăng lên 1.
2	gộp(2, 3)	[0, 0, 2, 2, 4, 5, 6]	[1, 0, 1, 0, 0, 0, 0]	Rank bằng nhau (0). Treo 3 dưới 2. Rank gốc 2 tăng lên 1.
3	gộp(1, 3)	[0, 0, 0, 2, 4, 5, 6]	[2, 0, 1, 0, 0, 0, 0]	Gốc của 1 là 0 (rank 1), gốc của 3 là 2 (rank 1). Treo 2 dưới 0. Rank gốc 0 tăng lên 2.
4	connected(0, 3)	[0, 0, 0, 0, 4, 5, 6]	[2, 0, 1, 0, 0, 0, 0]	True. Hàm tìm kiếm gốc của 3 đi qua 3 -> 2 -> 0. Nén đường đi kích hoạt: trở trực tiếp nút 3 về gốc 0.

5	gộp(4, 5)	[0, 0, 0, 0, 4, 4, 6]	[2, 0, 1, 0, 1, 0, 0]	Rank bằng nhau (0). Treo 5 dưới 4. Rank gốc 4 tăng lên 1.
6	connected(0, 5)	[0, 0, 0, 0, 4, 4, 6]	[2, 0, 1, 0, 1, 0, 0]	False. Gốc đại diện của 0 là 0, gốc đại diện của 5 là 4. Hai gốc khác nhau nên không cùng nhóm.
7	gộp(5, 6)	[0, 0, 0, 0, 4, 4, 4]	[2, 0, 1, 0, 1, 0, 0]	Gốc của 5 là 4 (rank 1), gốc của 6 là 6 (rank 0). Rank của 4 lớn hơn; treo 6 dưới 4.
8	gộp(3, 5)	[0, 0, 0, 0, 0, 4, 4]	[2, 0, 1, 0, 1, 0, 0]	Gốc của 3 là 0 (rank 2), gốc của 5 là 4 (rank 1). Rank của 0 lớn hơn; treo 4 dưới 0. Chỉ duy nhất liên kết của gốc 4 trở về 0.
9	connected(0, 6)	[0, 0, 0, 0, 0, 4, 0]	[2, 0, 1, 0, 1, 0, 0]	True. Hàm tìm kiếm gốc của 6 đi theo nhánh 6 -> 4 -> 0. Nên đường đi kích hoạt: trở trực tiếp nút 6 về gốc 0.

3.12. Phân tích mở rộng: Tối ưu hóa Union by Size (Gộp theo kích thước)

Bên cạnh kỹ thuật tối ưu hóa Union by Rank truyền thống, một hướng tiếp cận tương đương rất phổ biến trong thiết kế thực tế mang tên **Gộp theo kích thước (Union by Size)**. Kỹ thuật này thay thế việc theo dõi chiều cao bằng việc duy trì một mảng số nguyên `size[x]` tại mỗi nút gốc, đại diện cho tổng số lượng phần tử thực tế thuộc cây có gốc tại x.

Quy tắc gộp nhóm lúc này thay đổi như sau: khi tiến hành hợp nhất hai nhóm, chúng ta luôn luôn treo gốc của cây chứa ít phần tử hơn (kích thước nhỏ hơn) dưới gốc của cây chứa nhiều phần tử hơn (kích thước lớn hơn). Sau khi gộp, gốc mới tối cao sẽ có kích thước mới bằng tổng kích thước của cả hai nhóm cộng lại. Về mặt hiệu năng, kỹ thuật Union by Size cung cấp một đảm bảo toán học tương đương hoàn hảo với Union by Rank, không chế chiều cao tối đa của cây không vượt quá mức $O(\log n)$ và khi kết hợp với Path Compression cũng đạt tốc độ siêu hằng số cực hạn $O(\alpha(n))$. Tuy nhiên, Union by Size sở hữu một ưu điểm thực tiễn vượt trội: nó cho phép chúng ta truy xuất chính xác quy mô thành viên của nhóm chứa phần tử x bất kỳ chỉ trong thời gian hằng số $O(1)$ bằng cách gọi $\text{size}[\text{find}(x)]$, rất hữu ích cho các bài toán thực tế cần thống kê quy mô nhóm.

3.13. Các lỗi thường gặp khi triển khai Union-Find

- Lỗi gộp trực tiếp tham số đầu vào ($\text{parent}[a] = b$): Đây là lỗi cực kỳ nguy hiểm và phổ biến. Khi thực hiện gộp, lập trình viên quên không gọi find để tìm gốc của hai phần tử, mà tiến hành gán trực tiếp parent của a bằng b . Phép gán ngây thơ này lập tức cắt đứt liên kết của toàn bộ cây con đang treo dưới quyền nút a , chia cắt nhóm hiện tại thành hai phần độc lập một cách âm thầm, dẫn đến kết quả sai lệch nghiêm trọng trên các bộ dữ liệu lớn. Hãy luôn ghi nhớ quy tắc vàng: mọi phép hợp nhất phải được thực hiện trực tiếp trên hai nút gốc đại diện thu được từ hàm find , tuyệt đối không thao tác trực tiếp trên các tham số đầu vào.
- Làm tương hai tối ưu hóa có thể thay thế nhau: Một số lập trình viên cho rằng chỉ cần cài đặt một trong hai tối ưu hóa (hoặc Union by Rank, hoặc Path Compression) là đủ. Trên thực tế, chúng hoạt động ở hai thời điểm hoàn toàn độc lập: Rank phòng ngừa khi union, Compression sửa chữa khi find. Nếu chỉ dùng một tối ưu hóa đơn lẻ, chi phí cho mỗi thao tác bị đẩy về mức $O(\log n)$. Chỉ khi kết hợp đồng thời cả hai, hệ thống mới đạt được tốc độ siêu hằng số cực hạn $O(\alpha(n))$.

3.14. Ứng dụng thực tế của Union-Find

- Hợp nhất tài khoản người dùng trùng lặp trong hệ thống dữ liệu lớn (Account Merging): Khi hệ thống back-end phát hiện ra hai tài khoản khác nhau sở hữu chung thuộc tính định danh (như dùng chung số điện thoại hoặc tài khoản ngân hàng), hệ thống chỉ cần gọi $\text{union}(a, b)$. Việc kiểm tra hai tài khoản có thuộc cùng một người dùng hay không chỉ đơn giản là $\text{connected}(a, b)$. Nhờ đó, việc xác định thực thể gốc của một chuỗi hàng triệu liên kết diễn ra gần như lập tức, giúp hệ thống de-duplication vận hành trơn tru ở quy mô cực lớn mà không cần dịch chuyển dữ liệu vật lý.
- Thuật toán Kruskal tìm Cây khung nhỏ nhất (Minimum Spanning Tree - MST): Trong Kruskal, các cạnh đồ thị được duyệt theo thứ tự trọng số tăng dần. Với mỗi cạnh (u, v) , Union-Find giúp kiểm tra xem u và v có thuộc cùng một thành phần liên thông từ trước hay chưa. Nếu chưa (tức $\text{union}(u, v)$ trả về true), cạnh được chọn và đưa vào MST mà không sợ tạo thành chu trình đóng (Redundant Connection). Nhờ có Union-Find tối ưu, việc kiểm tra chu trình đạt tốc độ siêu hằng số cực hạn, giúp thuật toán Kruskal đạt hiệu năng thực tế tối đa $O(E \log E)$ bị chi phối chủ yếu bởi bước sắp xếp cạnh.

4. PHÂN TÍCH SO SÁNH VÀ ĐÁNH GIÁ ĐỐI SÁNH

4.1. Bài toán xử lý chuỗi và từ điển: Trie vs Hash Table vs Balanced BST

Để lượng hóa và chứng minh một cách khoa học các luận điểm lựa chọn, bảng dưới đây đối chiếu chi tiết hiệu năng và thuộc tính giữa các cấu trúc dữ liệu trong bài toán xử lý chuỗi và từ điển:

Tiêu chí so sánh	Trie (Cây tiền tố)	Bảng băm (Hash Table)	Cây BST cân bằng
Tìm kiếm chính xác	$O(L)$ - Worst-case	$O(L)$ - Trung bình (Hash cost)	$O(L \log n)$ - Rất tốn kém so sánh

Kiểm tra tiền tố (Prefix exists?)	$O(L)$ - Cực nhanh	$O(n * L)$ - Phải quét toàn bộ	$O(L \log n)$ - Duyệt theo thứ tự
Liệt kê từ có tiền tố P	$O(L + K)$ - K là kích thước kết quả	$O(n * L)$ - Không hỗ trợ tự nhiên	$O(L \log n + K)$ - Duyệt qua subtree
Duyệt theo thứ tự từ điển	Hỗ trợ tự nhiên (DFS duyệt cây)	Không hỗ trợ	Hỗ trợ tự nhiên (In-order traversal)
Chi phí bộ nhớ	Lớn nhất: $O(n * L * \Sigma)$	Thấp: $O(n * L)$	Moderate: $O(n * L)$
Ngữ cảnh tối ưu	Autocomplete, IP Routing, Spell Checker	Tìm kiếm chính xác (Exact Lookup)	Truy vấn khoảng (Range Query)

4.2. Bài toán liên thông đồ thị: Union-Find vs BFS/DFS

Bảng dưới đây so sánh hiệu năng và khả năng đáp ứng giữa Union-Find và các giải pháp duyệt đồ thị truyền thống trong bài toán kết nối động:

Tiêu chí	Union-Find (DSU tối ưu)	Duyệt đồ thị (BFS / DFS)
Phù hợp loại đồ thị	Đồ thị động (Liên tục thêm cạnh mới)	Đồ thị tĩnh (Cấu trúc ít thay đổi)
Kiểm tra liên thông $connected(u, v)$	$O(\alpha(n))$ khâu hao ($\sim O(1)$ thực tế)	$O(V + E)$ cho mỗi lần truy vấn
Chi phí chèn cạnh mới $union(u, v)$	$O(\alpha(n))$ khâu hao ($\sim O(1)$ thực tế)	$O(1)$ (Thêm vào danh sách kề)

Bộ nhớ phụ trợ	$O(n)$ (Chỉ gồm 2 mảng tính parent và rank)	$O(V + E)$ (Danh sách kề)
Khả năng truy vết đường đi cụ thể	Không hỗ trợ	Hỗ trợ đầy đủ (Truy vết các đỉnh đi qua)
Khả năng tách kết nối (Disconnect)	Không hỗ trợ trên phiên bản chuẩn	Hỗ trợ (Xóa cạnh và duyệt lại đồ thị)

4.3. So sánh triết lý thiết kế tổng thể giữa Trie và Union-Find

Dù có cấu trúc cơ học và cơ chế vận hành bộ nhớ hoàn toàn khác biệt, Trie và Union-Find thực chất đều được sinh ra từ một triết lý thiết kế hệ thống chung: *Chủ động từ bỏ các năng lực đa dụng để đổi lấy hiệu năng xử lý cực hạn cho một câu hỏi rất hẹp.* Bảng đối chiếu chi tiết triết lý kiến trúc giữa hai cấu trúc dữ liệu chuyên biệt này:

Thuộc tính so sánh	Cấu trúc Trie (Cây tiền tố)	Cấu trúc Union-Find (Disjoint-Set)
Câu hỏi hẹp được giải quyết	Những khóa nào trong từ điển bắt đầu bằng chuỗi tiền tố P cho trước?	Hai phần tử bất kỳ a và b có thuộc cùng một thành phần liên thông logic hay không?
Ý nghĩa của đường đi trên cây	Đường đi từ gốc (root) chính là biểu diễn vật lý của từ khóa (The path is the key).	Đường đi không mang ý nghĩa thông tin; nó hoàn toàn có thể bị thay đổi, làm phẳng tự do để tối ưu hiệu năng.
Năng lực bị chủ động từ bỏ	Từ bỏ sự tiết kiệm bộ nhớ (tiêu tốn RAM rất lớn cho các nút và liên kết con).	Từ bỏ khả năng liệt kê các thành viên trong một nhóm và từ bỏ khả năng xóa kết nối (undo).

Độ phức tạp thời gian vận hành	$O(L)$ phụ thuộc hoàn toàn vào độ dài chuỗi ký tự, độc lập với kích thước tập dữ liệu n .	$O(\alpha(n))$ khấu hao cho mỗi thao tác, tương đương hằng số thực tế trong thế giới vật lý.
--------------------------------	---	--

4.4. Khung ra quyết định kỹ thuật toàn diện (Technical Decision Framework)

Trong kỹ nghệ phần mềm thực tế, việc đưa một cấu trúc dữ liệu chuyên biệt vào hệ thống luôn đi kèm với cái giá phải trả về độ phức tạp của mã nguồn, rào cản đọc hiểu của đội ngũ lập trình, và chi phí bảo trì lâu dài. Do đó, một kỹ sư hệ thống chuyên nghiệp cần kiểm chứng nghiêm túc thông qua **Khung 3 câu hỏi sàng lọc (Framework of 3 Questions)** sau:

- 1. Mẫu truy vấn nào thực sự lặp đi lặp lại nhiều nhất và định hình hiệu năng hệ thống?** Nếu câu hỏi chỉ dừng lại ở mức tìm kiếm chính xác (Exact Lookup), bảng băm truyền thống luôn luôn là giải pháp tối ưu nhất. Chỉ khi nào câu hỏi mang tính đặc thù sâu sắc như tìm kiếm tiền tố (Prefix Queries) hoặc cập nhật liên kết động liên tục, chúng ta mới có lý do hợp lý để chuyển sang xem xét Trie hoặc Union-Find.
- 2. Hệ thống có thực sự sẵn sàng đánh đổi các khả năng bị từ bỏ của cấu trúc chuyên biệt đó hay không?** Mỗi cấu trúc chuyên biệt đều có một điểm mù lớn được đánh đổi chủ động để lấy hiệu năng cực hạn. Nếu chọn Trie, hệ thống có đủ dung lượng RAM để chịu đựng sự lãng phí tài nguyên của các nút con hay không? Nếu không, mảng sắp xếp kết hợp tìm kiếm nhị phân là sự lựa chọn khôn ngoan hơn. Tương tự, nếu chọn Union-Find, nghiệp vụ có đòi hỏi phải 'liệt kê danh sách thành viên' hay 'hoàn tác thao tác gộp' không? Nếu có, Union-Find sẽ dẫn đến các lỗi logic nghiêm trọng, buộc phải dùng đồ thị tổng quát.
- 3. Quy mô dữ liệu hiện tại có đủ lớn để lợi ích hiệu năng mang lại đáng giá với chi phí bảo trì?** Các phân tích độ phức tạp thời gian tiệm cận như $O(L)$ hay $O(\alpha(n))$ chỉ thực sự phát huy tác dụng thực tế khi kích thước dữ liệu vượt qua một ngưỡng giới hạn vật lý nhất định (thường là hàng trăm nghìn hoặc hàng triệu phần tử). Nếu

từ điển chỉ có vài trăm từ khóa tĩnh, việc thiết lập cấu trúc Trie phức tạp sẽ không mang lại bất kỳ sự cải thiện nào, ngược lại còn làm phức tạp hóa mã nguồn vô ích.

5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN MỞ RỘNG

5.1. Tóm tắt kết quả nghiên cứu

Báo cáo chuyên đề đã phân tích chuyên sâu hai cấu trúc dữ liệu chuyên biệt tiêu biểu là Trie (Cây tiền tố) và Union-Find (DSU). Kết quả nghiên cứu khẳng định cấu trúc chuyên biệt có giá trị tối thượng khi nó biến câu hỏi quan trọng nhất của workload thành thao tác tự nhiên trên representation, với chi phí bộ nhớ và tính linh hoạt được đánh đổi một cách có ý thức. Việc lấp đầy các khoảng trống hiệu năng của hộp công cụ cơ bản bằng các cấu trúc chuyên biệt chính là nấc thang chuyển đổi từ lập trình viên thông thường sang một kiến trúc sư phần mềm chuyên nghiệp.

5.2. Hướng phát triển mở rộng cho Trie

- Cây tiền tố nén (Radix Tree / Compressed Trie): Giải quyết điểm nghẽn lãng phí bộ nhớ bằng cách nén các chuỗi nút chỉ có duy nhất một nút con thành một nút duy nhất lưu đoạn chuỗi dài (substring), giúp tiết kiệm tới 60% dung lượng RAM của Trie thông thường.
- Cây hậu tố (Suffix Tree / Suffix Array): Cấu trúc dữ liệu chuyên sâu nâng cao hỗ trợ tìm kiếm chuỗi con (substring search) bất kỳ trong thời gian tuyến tính, đóng vai trò xương sống cho các hệ thống đối chiếu chuỗi DNA và xử lý dữ liệu sinh học.

5.3. Hướng phát triển mở rộng cho Union-Find

- Union-Find hỗ trợ hoàn tác (DSU with Rollback): Sử dụng cấu trúc ngăn xếp (Stack) để lưu lại lịch sử thay đổi của mảng parent thay vì sử dụng Path Compression, cho phép hủy bỏ thao tác union gần nhất trong thời gian $O(\log n)$, hỗ trợ giải quyết các bài toán đồ thị động phức tạp cho phép xóa cạnh.
- Persistent DSU: Duy trì các phiên bản lịch sử của rừng cây, cho phép truy vấn trạng thái liên thông của hệ thống tại bất kỳ thời điểm nào trong quá khứ.

6. TÀI LIỆU THAM KHẢO (REFERENCES)

- [1] E. Fredkin, “Trie Memory,” *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960.
- [2] D. R. Morrison, “PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric,” *Journal of the ACM*, vol. 15, no. 4, pp. 514–534, 1968.
- [3] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [4] R. E. Tarjan and J. van Leeuwen, “Worst-Case Analysis of Set Union Algorithms,” *Journal of the ACM*, vol. 31, no. 2, pp. 245–281, 1984.
- [5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley Professional, 2011.
- [6] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [7] T-Bao, “Specialized Structures: Some Notes,” *MinBook*, Kmin Academy of Computer Science, 1st ed., 2026.
- [8] S. Jayanti, R. E. Tarjan, and E. Boix-Adserà, “Randomized Concurrent Set Union and Generalized Wake-Up,” *Proc. ACM PODC*, 2019.
- [9] MIT OpenCourseWare, “Disjoint-set Forest,” 6.046J Design and Analysis of Algorithms, Spring 2012.
- [10] CP-Algorithms, “Disjoint Set Union,” implementation-oriented algorithm reference, updated 2026.